In this section, we're going to look at ways to set *constraints* on a test double, so that RSpec only uses it if your code calls it in a certain way.

Constraining Arguments

In your projects, you'll often want to check that your code is calling a method with the correct parameters. To constrain what arguments your mock object will accept, add a call to with to your message expectation:

14-customizing-test-doubles/02/setting_constraints.rb

```
expect(movie).to receive(:record_review).with('Great movie!')
expect(movie).to receive(:record_review).with(/Great/)
expect(movie).to receive(:record_review).with('Great movie!', 5)
```

If your code calls the method with arguments that don't match the constraint, then the expectation remains unsatisfied. RSpec will treat it the same as any other unmet expectation. In this example, we're using expect, meaning that RSpec will report a failure:

```
>> expect(movie).to receive(:record_review).with('Good')
=> #<RSpec::Mocks::MessageExpectation #<Double "Jaws">.record_review("Good")>
>> movie.record_review('Bad')
RSpec::Mocks::MockExpectationError: #<Double "Jaws"> received                ↩
:record_review with unexpected arguments
  expected: ("Good")
       got: ("Bad")
    « backtrace truncated »
```

If we had used allow instead, RSpec would have looked for another expectation that fit the passed-in arguments:

```
>> allow(imdb).to receive(:rating_for).and_return(3) # default
=> #<RSpec::Mocks::MessageExpectation #<Double "IMDB">.rating_for(any         ↩
arguments)>
>> allow(imdb).to receive(:rating_for).with('Jaws').and_return(5)
=> #<RSpec::Mocks::MessageExpectation #<Double "IMDB">.rating_for("Jaws")>
>> imdb.rating_for('Weekend at Bernies')
=> 3
>> imdb.rating_for('Jaws')
=> 5
```

RSpec gives you a ton of ways to constrain method arguments. Your test doubles can require something as simple as a specific value, or as sophisticated as any custom logic you can devise.